

Day 35 — Fri 9 Oct

Countdown: 1 day to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

☉ Degree — mobile & web/PHP

Source: `ComputerScience_Degree/CSD_04_MobileApplicationsAndWeb.md`

CSD_04 — Mobile Applications & Services + Web Technologies

Paper-I, Unit 6 — "Mobile Application & Services" and **Paper-II, Unit 5 — "Web Technologies"**, taken together in one file because they are really one story: building a piece of software that a user touches, whether it lives on a phone or in a browser.

Official syllabus wording, Paper-I §6: *"Factors in Developing Mobile Applications, Frameworks and Tools in Mobile Applications, Text-to-Speech Techniques, Designing the Right UI, Multichannel and Multimodal UIs, Storing and Retrieving Data, Synchronization and Replication of Mobile Data, Android Storing and Retrieving Data, Putting It All Together: Packaging and Deploying."*

Official syllabus wording, Paper-II §5: *"The Phases of web site development: Implementation, Maintenance, Testing. Basic HTML Concepts, HTML, HEAD, TITLE, BODY, Paragraphs, Lists, Formatted and Unformatted Text, Hyperlink, Font (Size, Colour), image. PHP: Server-side web scripting, Installing PHP, Adding PHP to HTML, Syntax and Variables, Passing information between pages, Basic PHP error/problems."*

Networking mechanics (OSI, TCP/IP, IP addressing) are covered in `CORE_05_ComputerNetworks.md` — this file only covers the client-server HTTP cycle as it applies to a website request, not network layering. Nothing here overlaps `CORE_09_ProgrammingC_CPP.md` either — PHP is a different language, taught from scratch below.

Why this matters / where it appears

1. **Both halves are guaranteed, standalone, low-competition units.** Nothing else in the CS Degree syllabus touches mobile development or PHP/HTML — this is 100% new content, which means it is *entirely* yours to control, unlike, say, OS scheduling which nine other candidates also know cold.
2. **The 2025 papers hit both halves hard, in prose-answer form, not MCQ trivia.** Paper-I carried a 15-mark "frameworks and tools in mobile app development" question and a 15-mark "designing the right UI for mobile" question — that is 30 of 100 Paper-I marks from this one unit alone. Paper-II carried a 10-mark "GET vs POST" question, a 10-mark "PHP syntax and variables" question, and a 15-mark "phases of website development" question.
3. **The examiner wants explanatory prose, not bullet trivia.** Look at the 2025 answer style: full paragraphs that reason from constraints to conclusions ("the screen is small, the input is a fingertip, so touch targets must be at least 48×48dp, so..."). MCQ-style fact lists lose marks here. Practice **writing**, not just reading, the paragraphs below.
4. **PHP/HTML code must actually run in your head.** Every code block below has its output traced by hand — do the same for anything you write in the exam. A `$_GET` vs `$_POST` mix-up is a guaranteed trap question.

Factor	The constraint	What it forces the developer to do
Battery	Finite, non-renewable during use	Minimise CPU wake-ups, GPS polling, network radio use; batch network calls; use push instead of poll
Network	Intermittent, slow, metered, expensive on roaming	Design for offline-first ; cache aggressively; compress payloads; retry with backoff
Screen size & density	From a 4" phone to a 12" tablet, at wildly different pixel densities	Responsive/adaptive layout; density-independent units (dp/pt), not raw pixels
Input method	Touch (imprecise, no hover, multi-touch), not mouse+keyboard	Larger touch targets; gestures instead of right-click menus
Processing power & memory	Weaker CPU, less RAM, no swap space of consequence	Lightweight data structures; avoid memory leaks (OS kills the app under pressure)
Fragmentation	Many OS versions, many manufacturers, many screen shapes in the wild simultaneously	Test across a device/OS matrix; use compatibility libraries
Interruptions	Calls, notifications, screen lock, app switch, at any moment	The app must save state and resume gracefully — this is <i>the</i> reason for the Android activity lifecycle (§7)
Context of use	Outdoors in glare, one-handed, walking, distracted	Short tasks, high contrast, forgiving touch targets
App store review & distribution	Gatekept release channel, versioned rollout	Build, sign and package correctly before every release (§9)
Security	Device can be lost, stolen, rooted; app runs on hardware the developer does not control	Encrypt local data; do not trust client-side validation alone
Multiple platforms	iOS, Android, sometimes web, side by side	Choose native / hybrid / cross-platform trade-off (§2) up front — expensive to change later

Likely exam questions

1. Discuss the various factors that must be considered when developing a mobile application. **[15]** (2025 pattern)
2. Why does mobile application design differ fundamentally from desktop software design? **[10]**
3. Explain how battery and network constraints influence mobile app architecture. **[5]**

MCQ traps

- "Fragmentation" in mobile computing means device/OS diversity, **not** file/memory fragmentation (that is an OS-unit term — do not confuse across units).
- Density-independent units are **dp** (Android) / **pt** (iOS), not raw pixels.

2. Native vs hybrid vs web apps; frameworks and tools

Concept — in plain English first

There are three ways to put an app on somebody's phone, and the difference is simply: **which language do you write in, and does the OS know your code exists as a "real" app or not.**

Key points — the three approaches

	Native	Hybrid	Web / PWA
Language	Platform's own (Kotlin/Java on Android, Swift/Obj-C on iOS)	Cross-platform framework compiling/bridging to native (React Native, Flutter)	HTML/CSS/JavaScript, runs in a browser or wrapped in one (Ionic + Capacitor)
Performance	Best — direct OS/hardware access	Good — near-native (Flutter compiles to ARM) to good (RN bridges to native widgets)	Weakest — runs inside a browser engine
Code reuse across platforms	None — separate codebase per OS	High — one codebase, most logic shared	Highest — one codebase, no app-store build at all
Access to device hardware (camera, sensors, contacts)	Full , immediate on every OS release	Good, via plugins; may lag behind new OS APIs	Limited, browser-API dependent
Distribution	App store only	App store (packaged)	App store optional; can be a plain URL
Look and feel	Matches OS conventions automatically	Matches if using platform-styled widgets	Must be hand-matched; often feels "web-like"
Development cost	Highest (2x team/skillset for 2 platforms)	Medium	Lowest
Example frameworks	Android SDK + Android Studio, Xcode + Swift	React Native (JavaScript, bridges to native), Flutter (Dart, compiles to native ARM code, own rendering engine, strong hot reload)	Ionic + Capacitor (web app in HTML/CSS/JS wrapped in a native shell)

Frameworks and tools to be able to name (this is what the 2025 15-marker rewarded):

Category	Tools
IDEs	Android Studio (official Android IDE, Gradle build system), Xcode (iOS)
Cross-platform frameworks	React Native, Flutter, Ionic/Capacitor, Xamarin
Local storage	SQLite, Room (Android's SQLite abstraction), Core Data (iOS), Realm, key-value stores
UI guideline systems	Material Design (Android), Human Interface Guidelines (iOS)
Testing	JUnit and Espresso (Android), XCTest (iOS), Appium and Detox (cross-platform end-to-end)
Debugging/monitoring	Android Studio Profiler, Xcode Instruments, Flipper, Sentry/Crashlytics (crash reporting)

Choosing between them — the one-paragraph answer: native wins when performance or full hardware access is critical (games, camera-heavy apps); hybrid/cross-platform wins when time-to-market and one shared codebase across iOS+Android matter more than squeezing out the last bit of performance; a web app/PWA wins when you need zero-install reach and app-store approval is undesirable or too slow.

Likely exam questions

1. Discuss the different frameworks and tools commonly used in mobile application development. [15]
(*exactly the 2025 question*)
2. Differentiate between native, hybrid and web-based mobile applications, with examples of each. [10]
3. Compare React Native and Flutter as cross-platform frameworks. [5]

MCQ traps

- Flutter compiles to **native ARM code**; React Native **bridges** JavaScript to native widgets — these are different mechanisms, commonly swapped in MCQs.
 - Ionic/Capacitor apps are fundamentally **web apps** wrapped in a native shell, not compiled native code.
-

3. Text-to-speech techniques

Concept — in plain English first

Text-to-speech (TTS) converts written text into spoken audio, so the app can be used hands-free/eyes-free (driving, accessibility for the visually impaired, notifications read aloud).

Key points

Stage	What happens
Text analysis / normalisation	Expand abbreviations, numbers, dates into full words ("Dr." → "Doctor", "12/5" → "the twelfth of May")
Linguistic analysis	Determine pronunciation, word stress, sentence intonation (prosody)
Waveform synthesis	Two main techniques:
— Concatenative synthesis	Stitches together pre-recorded snippets of real human speech; natural-sounding but limited to recorded vocabulary/voice
— Parametric / formant synthesis	Generates speech from a mathematical model of the vocal tract; more flexible (any text, any speed/pitch), can sound more robotic
— Neural TTS (modern)	Deep learning models (e.g. WaveNet-style) generate waveforms directly; most natural-sounding, used in modern assistants
Platform APIs	Android: <code>TextToSpeech</code> class; iOS: <code>AVSpeechSynthesizer</code> ; Web: <code>SpeechSynthesis</code> Web API

Likely exam questions

1. Explain the working of text-to-speech systems in mobile applications. [5]
2. Differentiate between concatenative and parametric speech synthesis. [5]

MCQ traps

- TTS is **text** → **speech**; speech recognition (STT / ASR) is the reverse — do not swap them.
-

4. Designing the right UI for mobile

Concept — in plain English first

Mobile UI design is the discipline of working *within* the constraints from §1, not fighting them. Every rule below traces back to one of: small screen, imprecise touch input, interruption-prone usage, or platform convention.

Key points

Constraint	Design rule that follows
Screen fragmentation (thousands of sizes/densities across Android; notches and safe areas on iOS)	Use responsive/constraint-based layout and density-independent units (dp/pt), not fixed pixels
Fingertip imprecision	Minimum touch target 48×48 dp (Android) / 44×44 pt (iOS) , adequate spacing between targets
Platform convention	Follow Material Design (Android) or Human Interface Guidelines (iOS) — users expect platform-native navigation patterns (back button behaviour, tab bars)
Interrupted, one-handed, outdoors, in-motion usage	Short tasks; large legible text; high contrast; avoid tasks that need two hands or sustained attention
Limited screen real estate	Progressive disclosure (show only what is needed now); avoid dense desktop-style menus
Users get lost easily on small screens	Clear visual hierarchy; the next action should be obvious, reachable (thumb zone) and reversible (undo, back)

The one-line thesis to open the essay with: *"Mobile UI design succeeds when the interface makes the next action obvious, reachable and reversible — every specific rule (touch target size, platform convention, short tasks) is a consequence of that one goal under the constraints of a small screen and an imprecise, interruptible way of interacting with it."*

Likely exam questions

1. Discuss the challenges and considerations involved in designing the right UI for mobile applications. [15]
(*exactly the 2025 question*)
2. What is the minimum recommended touch target size, and why? [5]
3. Compare Material Design and Human Interface Guidelines as UI convention systems. [5]

MCQ traps

- Minimum touch target: **48×48 dp** Android, **44×44 pt** iOS — these numbers, and which unit belongs to which OS, are MCQ-tested.

5. Multichannel and multimodal UIs

Concept — in plain English first

Multichannel = the same service delivered over more than one *channel* (app, SMS, web, voice IVR, in-branch kiosk) — the user picks the channel; think of a bank offering mobile app, website and phone-banking IVR for the same account.

Multimodal = a *single* interaction that supports more than one input/output *modality* simultaneously — voice + touch + gesture in the same interface, e.g. saying "call mom" while also being able to tap a contact.

	Multichannel	Multimodal
Axis of variation	Channel (delivery medium)	Modality (interaction mode)
Example	Same banking service via app, SMS, web, IVR	Voice assistant that also accepts touch and gesture in one session
Design challenge	Consistency of data/state across channels	Fusing/disambiguating simultaneous inputs

Likely exam questions

1. Distinguish between multichannel and multimodal user interfaces, with examples. [10]
2. Why is consistency across channels important in a multichannel system? [5]

MCQ traps

- "Multichannel" and "multimodal" are the single most confused pair of terms in this unit — channel = *where*, modality = *how*.

6. Storing and retrieving data; synchronisation and replication

Concept — in plain English first

A mobile device is frequently offline, so an app cannot simply talk to a central server for every read/write the way a web app can. It needs **local storage** for when offline, and a **sync strategy** to reconcile local changes with the server once connectivity returns.

Key points

Local storage options (platform-agnostic):

Option	Best for
Key-value store (SharedPreferences/UserDefaults)	Small settings, flags
Embedded relational DB (SQLite / Room / Core Data)	Structured, queryable data
Object/NoSQL store (Realm)	Object-graph data, offline-first apps
Flat files	Media, large blobs, exported data

Synchronisation and replication:

Concept	Meaning
---------	---------

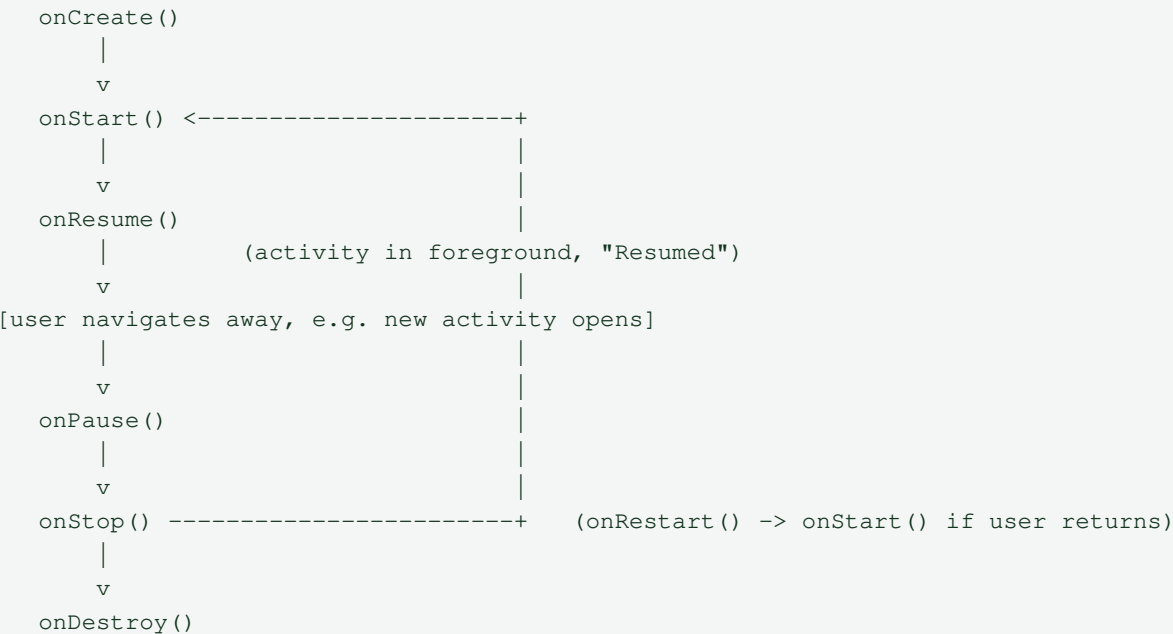
+-----+

Draw this as five stacked horizontal bands, labelled bottom-to-top: **Linux Kernel** → **HAL** → **Libraries/ART** → **Application Framework** → **Applications**.

7.2 The four Android application components

Component	Role
Activity	One screen with a UI the user interacts with
Service	Runs in the background, no UI (e.g. playing music, downloading)
Broadcast Receiver	Responds to system-wide or app broadcast events (e.g. battery low, SMS received)
Content Provider	Manages a structured, shared set of app data, exposing it to other apps via a standard interface

7.3 The activity lifecycle — the state machine to draw



Callback	Fired when	Typical use
<code>onCreate()</code>	Activity is first created	One-time setup: inflate layout, bind views
<code>onStart()</code>	Activity becomes visible	
<code>onResume()</code>	Activity gains foreground focus, user can interact	Start animations, camera preview, sensors
<code>onPause()</code>	Another activity partially/fully covers this one	Release resources shared with other apps quickly (camera), save lightweight state
<code>onStop()</code>	Activity no longer visible at all	Release heavier resources
<code>onRestart()</code>	Stopped activity is coming back to the foreground	

<code>onDestroy()</code>	Activity is finishing, or OS is reclaiming memory	Final cleanup
--------------------------	---	---------------

Why the lifecycle exists at all — the answer that gains marks: it exists *because* of the mobile constraint from §1 — interruptions (calls, notifications, app switching) can end an app's foreground time at any moment, and the device has limited memory so the OS may kill backgrounded processes without warning. The lifecycle gives the app defined callback points to save state before that happens, and to restore it cleanly when the user returns.

7.4 Intents

An **Intent** is a messaging object used to request an action from another app component.

Type	Meaning	Example
Explicit intent	Names the exact target component (by class)	Launching a specific activity within your own app
Implicit intent	Declares a general action; the OS resolves which app/component can handle it	"Share this text" → OS offers all apps that registered to handle sharing

Intents also carry **extras** (key-value data payload) and can request a result back (`startActivityForResult`, or the modern Activity Result API).

7.5 Android storage options (the syllabus explicitly names this: "Android Storing and Retrieving Data")

Option	What it stores	Persistence scope
SharedPreferences	Small key-value pairs (settings, flags)	App-private, survives app restarts
Internal storage	Files private to the app	Deleted when app is uninstalled
External storage	Files on shared/removable storage	May be readable by other apps (with permission)
SQLite database (via Room)	Structured relational data	App-private, persists until uninstall or explicit clear
Network / cloud storage	Remote data, fetched and cached	Not local; requires sync strategy (§6)

7.6 The Android Manifest

`AndroidManifest.xml` is the app's declaration file, required in every Android project. It declares:

- All four components used by the app (activities, services, receivers, providers) — components not declared here cannot run.
- The app's **permissions** (camera, internet, location, ...) requested from the user/OS.
- The **minimum and target SDK versions**.
- The app's **package name**, version code/name.
- Hardware/software features required (e.g. camera, GPS).
- The **launcher activity** (which activity opens when the user taps the app icon), marked with an intent-filter for `MAIN/LAUNCHER`.

Likely exam questions

1. Describe the Android architecture stack with the help of a diagram. [10]
2. Explain the Android activity lifecycle with a diagram. Why is it necessary? [10]
3. What are the four fundamental components of an Android application? Explain each briefly. [10]
4. Differentiate between explicit and implicit intents, with an example of each. [5]
5. What is the role of the Android Manifest file? List four things it declares. [5]
6. Describe Android's data storage options and when each is appropriate. [10]

MCQ traps

- The four Android components: **Activity, Service, Broadcast Receiver, Content Provider** — a fifth fake option ("Intent Filter", "Fragment") is a common distractor; Fragment is a UI sub-component, not one of the four.
- `onPause()` vs `onStop()`: pause = partially/fully covered but potentially visible; stop = fully invisible. Order is always Pause → Stop, never reversed.
- An implicit intent may match **zero, one, or many** apps; the OS shows a chooser if more than one.
- The Manifest file is `AndroidManifest.xml`, not `.json` or `.properties`.

8. Packaging and deploying: APK, signing, Play Store

Concept — in plain English first

Writing the code is only half the job — "Putting It All Together" (the syllabus's own phrase) is the pipeline that turns source code into something installable and distributable.

Key points

Step	What happens
1. Build/compile	Source (Kotlin/Java) + resources compiled into <code>.dex</code> bytecode
2. Package into APK	The APK (Android Package) bundles compiled code, resources, assets and the manifest into one archive; AAB (Android App Bundle) is the modern Play-Store-preferred format that lets Google Play generate optimised APKs per device
3. Sign	Every APK/AAB must be digitally signed with a developer's private key before it can be installed on a device or accepted by a store. Signing proves the app's identity and that updates come from the same author — Android refuses to install an update signed with a different key than the original
4. Test build variants	Debug build (unsigned or debug-signed, for development) vs release build (signed with the production key, optimised/minified)
5. Versioning	<code>versionCode</code> (integer, must increase for every Play Store update) and <code>versionName</code> (human-readable, e.g. "2.3.1")
6. Upload & listing	Upload the signed AAB/APK to the Play Console; provide store listing (title, description, screenshots, privacy policy, content rating)

7. Review	Google Play reviews the app for policy compliance before publishing
8. Staged rollout	Release to a percentage of users first, monitor crash reports, then expand
9. Updates	New versions must be re-signed with the same key and a higher <code>versionCode</code>

Why signing matters enough to be its own exam point: it is the mechanism that lets the OS and the store trust that an "update" to an app genuinely comes from the original developer and has not been tampered with — losing your signing key means you can never again publish an update to the same app listing.

Likely exam questions

1. Describe the process of packaging and deploying an Android application, from build to Play Store. **[10]**
2. Why must an APK be digitally signed? What happens if a signing key is lost? **[5]**
3. Differentiate between `versionCode` and `versionName`. **[5]**

MCQ traps

- APK = **Android Package**, the installable archive; AAB = **Android App Bundle**, the upload format Play Store now prefers.
- An app **cannot** be installed/updated on a device without a valid signature.
- `versionCode` must strictly **increase** on every Play Store submission; `versionName` is just a display string with no such constraint.

PART B — WEB TECHNOLOGIES

9. Phases of website development

Concept — in plain English first

Building a website has a lifecycle shaped like the general SDLC (`CORE_07_SoftwareEngineering.md`) but with web-specific emphases the syllabus explicitly names: **Implementation, Maintenance, Testing** — plus the planning/design work that necessarily precedes them.

Key points

Phase	What happens
1. Planning / requirements	Define purpose, audience, content scope, budget, timeline
2. Design	Wireframes, visual design, information architecture; design responsively for mobile/tablet/desktop widths; design interactive states (hover, active, disabled)
3. Implementation	Front end: mark up pages semantically in HTML, style with CSS, add behaviour with JavaScript. Back end: build server logic and data storage using a stack such as PHP + MySQL, Node.js, Django or Laravel

4. Testing	Functional testing (links, forms work), cross-browser/cross-device testing, performance testing, security testing (e.g. against SQL injection, XSS)
5. Deployment	Deploy to a live web server / hosting provider, configure domain and DNS
6. Maintenance	Ongoing: content updates, security patches, bug fixes, monitoring uptime

The essential difference from general software development, worth stating in the answer: *a website is never "finished" — maintenance and iteration are continuous*, unlike a shrink-wrapped desktop application with discrete release cycles. Content changes, security patches, and browser/device drift all force ongoing work after "launch."

A commonly under-budgeted item, worth naming: cross-browser/cross-device testing — different browsers render subtly differently, and this is a very common cause of last-minute delay in a website project.

Likely exam questions

1. Describe the phases of website development, from planning to maintenance. **[15]** (*exactly the 2025 question*)
2. How does website development differ from general software development in terms of its lifecycle? **[5]**
3. What testing considerations are specific to websites, as opposed to desktop software? **[10]**

MCQ traps

- The three phases the syllabus explicitly names are **Implementation, Maintenance, Testing** — if a question asks "which three phases does the syllabus emphasise", these three (not "planning" or "design") are the literal answer.

10. HTML in detail

Concept — in plain English first

HTML is a **markup language**, not a programming language — it describes the *structure and meaning* of content (this is a heading, this is a list, this is a link), and the browser decides how to render that structure. Every tag has a **start tag**, optional **content**, and an **end tag**: `<tag>content</tag>`. Some tags are **void elements** with no content/end tag (`<img>`, `<br>`, `<hr>`).

10.1 Basic document structure

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph of unformatted text.</p>
</body>
</html>
```

10.5 Font (size, colour)

```
<font size="5" color="blue">Old-style font tag (deprecated in HTML5)</font>
<p style="font-size:20px; color:blue;">Modern way: inline CSS</p>
```

Exam point: `<font>` is deprecated in HTML5; the correct modern approach is CSS (`font-size`, `color` properties) — but the syllabus explicitly lists "Font (Size, Colour)" so know both the legacy tag and the modern replacement.

10.6 Images

```

```

`src` = source path/URL (required). `alt` = alternate text, shown if the image fails to load and read by screen readers (accessibility). `width/height` in pixels.

10.7 Tables

```
<table border="1">
  <tr>
    <th>Name</th><th>Marks</th>
  </tr>
  <tr>
    <td>Asha</td><td>85</td>
  </tr>
  <tr>
    <td>Toshi</td><td>90</td>
  </tr>
</table>
```

Tag	Role
<table>	Wraps the whole table
<tr>	Table row
<th>	Header cell (bold, centred by default)
<td>	Data cell
colspan / rowspan (attributes on <td>/<th>)	Merge cells across columns/rows

10.8 Forms

```
<form action="submit.php" method="post">
  Name: <input type="text" name="uname"><br>
  Password: <input type="password" name="pwd"><br>
  Gender:
  <input type="radio" name="gender" value="M">Male
  <input type="radio" name="gender" value="F">Female<br>
  Subjects:
  <input type="checkbox" name="subj[]" value="Maths">Maths
  <input type="checkbox" name="subj[]" value="Sci">Science<br>
  Country:
```

```
<select name="country">
  <option value="IN">India</option>
  <option value="US">USA</option>
</select><br>
<textarea name="comments" rows="4" cols="30"></textarea><br>
<input type="submit" value="Submit">
</form>
```

action = URL the form data is sent to. **method** = **get** or **post** (see §14 — this is the exact 2025 exam pairing of concepts: HTML forms in this section, GET vs POST in the PHP section). Every input needing to reach the server **must have a name attribute** — inputs without **name** are never submitted.

Likely exam questions

1. Explain the basic structure of an HTML document with a diagram/example. [5]
2. Write HTML to create an ordered list, an unordered list and a definition list. [5]
3. Explain hyperlinks in HTML with examples of absolute, relative and anchor links. [5]
4. Write HTML to create a table with a header row and merged cells (colspan/rowspan). [10]
5. Write an HTML form containing a text box, radio buttons, a checkbox, a dropdown and a submit button. Explain the role of the **action** and **method** attributes. [10]
6. Differentiate between the **** tag and CSS for controlling text appearance. [5]

MCQ traps

- ****, **
, **<hr>, **<input>** are **void elements** — no closing tag.
- An **<a>** tag's destination attribute is **href**, **not src** (that is for ****).
- Form controls without a **name** attribute are **not sent** to the server on submit.
- **<th>** is bold/centred by **default rendering**, not because of any required CSS.

11. CSS basics

Concept — in plain English first

If HTML is the skeleton (structure/meaning), **CSS (Cascading Style Sheets) is the skin** — it controls colour, layout, spacing and typography, separately from the content.

Key points

Three ways to apply CSS:

```
<!-- 1. Inline -->
<p style="color:red;">Inline styled text</p>

<!-- 2. Internal (in <head>) -->
<style>
  p { color: red; font-size: 16px; }
</style>

<!-- 3. External (best practice) -->
<link rel="stylesheet" href="style.css">
```

Selectors:

Selector	Matches
<code>p</code>	Every <code><p></code> element
<code>.classname</code>	Every element with <code>class="classname"</code>
<code>#idname</code>	The single element with <code>id="idname"</code>
<code>p.warning</code>	A <code><p></code> that also has class <code>warning</code>

The cascade / specificity, in increasing priority: browser default → external/internal stylesheet (by selector specificity: element < class < id) → inline style → `!important`.

The box model — every HTML element is a box: `content` → `padding` → `border` → `margin` (innermost to outermost). This is the single most-asked CSS diagram — draw four nested rectangles labelled margin (outermost), border, padding, content (innermost).

Likely exam questions

1. What is CSS? Explain the three ways of applying CSS to an HTML page. [5]
2. Explain the CSS box model with a diagram. [5]
3. Differentiate between class and id selectors in CSS. [5]

MCQ traps

- `id` selectors should be **unique per page**; `class` selectors can repeat.
- Inline style has **higher** priority than an external/internal stylesheet rule of the same specificity.

12. PHP: server-side scripting

Concept — in plain English first

Two categories of "web scripting" exist, and the syllabus wants you to know the difference before anything else:

	Client-side	Server-side
Runs where	In the user's browser	On the web server, before the page is sent
Example languages	JavaScript	PHP , Python (Django), Node.js, Java (Servlets)
Can access the database directly?	No (would expose credentials)	Yes
Output the browser receives	The script itself, plus its effect on the page	Only the resulting HTML — the PHP source is never sent to the browser
Typical use	Form validation, animation, interactivity without a round trip	Generating dynamic HTML, database access, authentication, session/cookie logic

The one sentence that defines PHP: *PHP is a server-side interpreted scripting language — the server executes the PHP code and sends only the resulting HTML output to the browser; the client*

never sees the PHP source.

12.1 Installing PHP / adding PHP to HTML

- Install a PHP runtime (XAMPP/WAMP bundle PHP + Apache + MySQL together for local development, or install PHP standalone alongside a web server).
- The file extension **must be** `.php` — the web server hands `.php` files to the PHP interpreter; a `.html` file is served as-is and any PHP inside it is ignored (sent to the browser as literal text).
- PHP is embedded directly inside HTML using the tags:

```
<!DOCTYPE html>
<html>
<body>
<h2>Marks Report</h2>
<?php
    echo "Hello, World!";
?>
</body>
</html>
```

Anything **outside** `<?php ... ?>` is sent to the browser unchanged; anything inside is executed on the server. In a file that is pure PHP, the closing `?>` is conventionally **omitted** to avoid accidentally emitting trailing whitespace/newlines to the browser.

12.2 Syntax, variables, types

```
<?php
    $name = "Asha";           // variable: always starts with $, no type declaration
                              needed
    $marks = 85;              // integer
    $percentage = 85.5;       // float
    $passed = true;           // boolean
    $subject = null;          // null

    echo "Name: " . $name . ", Marks: " . $marks; // '.' concatenates strings
?>
```

Output: Name: Asha, Marks: 85

Rule	Detail
Variable naming	Must start with <code>\$</code> , then a letter or underscore, then letters/digits/underscores
Typing	Dynamically typed — no declaration, type is inferred from the assigned value, and can change at runtime
Case sensitivity	Variable names are case-sensitive (<code>\$Name</code> $\neq$ <code>\$name</code>); function/keyword names are not
Comments	<code>//</code> or <code>#</code> single line, <code>/* ... */</code> multi-line
Variable variables	<code>\$\$a</code> refers to the variable whose name is the <i>value</i> of <code>\$a</code> — e.g. if <code>\$a = "marks"</code> , then <code>\$\$a</code> is the same as <code>\$marks</code>

Data types: integer, float (double), string, boolean, array, object, NULL, resource.

Operators:

Category	Examples
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> (exponentiation)
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>
Comparison	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code> , and <code>===</code> (identical — value and type)
Logical	<code>&&</code> <code>\ </code> <code>!</code> (also <code>and</code> , <code>or</code>)
String	<code>.</code> (concatenation), <code>.=</code> (concatenate-assign)
Increment/decrement	<code>++</code> <code>--</code>

`==` vs `===` — a classic PHP MCQ: `"5" == 5` is **true (loose comparison, type-juggled);**

`"5" === 5` is **false (strict — string vs integer are different types).**

12.3 Control structures

```
<?php
    $marks = 72;
    if ($marks >= 90) {
        echo "Grade A";
    } elseif ($marks >= 60) {
        echo "Grade B";
    } else {
        echo "Grade C";
    }
?>
```

Output: `Grade B`

```
<?php
    for ($i = 1; $i <= 5; $i++) {
        echo $i . " ";
    }
?>
```

Output: `1 2 3 4 5`

Also available: `while`, `do...while`, `switch`, `foreach` (for arrays), `break`, `continue`.

12.4 Functions

```
<?php
    function average($a, $b) {
        return ($a + $b) / 2;
    }
    echo average(80, 90);
?>
```

Output: `85`

PHP also supports default parameter values, variable-length argument lists, and pass-by-reference

(`function inc(&$x) { $x++; }`).

12.5 Arrays

```
<?php
    $subjects = array("Maths", "Science", "English");    // indexed array
    echo $subjects[1];                                // Science

    $marks = array("Maths" => 85, "Science" => 90);      // associative array
    echo $marks["Science"];                            // 90

    foreach ($marks as $subject => $mark) {
        echo "$subject: $mark\n";
    }
?>
```

Output:

```
Science
90
Maths: 85
Science: 90
```

Likely exam questions

1. Differentiate between client-side and server-side scripting, with examples. **[5]**
2. What are the fundamental syntax rules and the use of variables in PHP scripting? **[10]** (*exactly the 2025 question*)
3. Explain PHP data types, operators and control structures with examples. **[10]**
4. Write a PHP script to find and display the largest of three numbers. **[10]**
5. Explain indexed and associative arrays in PHP with an example of each. **[5]**
6. Trace the output of a given PHP script (loops/conditionals). **[5/10]**

MCQ traps

- PHP files **must** use the `.php` extension for the server to invoke the interpreter.
 - `==` is loose (type-juggling) comparison; `===` requires **same value and same type**.
 - PHP variables need **no** type declaration and **no** semicolon-terminated type keyword — only `$name = value;`.
 - The closing `?>` is optional and conventionally **omitted** in pure-PHP files.
-

13. Passing information between pages: GET vs POST, sessions, cookies, form handling

Concept — in plain English first

HTTP itself is **stateless** — the server has no memory of who you are between one request and the next. Everything in this section exists to work around that one fact: GET/POST are ways to *send* data on a single request; sessions/cookies are ways to make the server *remember* you across many requests.

13.1 GET vs POST

```
<form action="process.php" method="get">
  Name: <input type="text" name="uname">
  <input type="submit">
</form>
```

Submitting "Asha" sends the browser to:
`process.php?uname=Asha` — data appended visibly to the URL as a query string.

```
<?php
  echo "Hello, " . $_GET["uname"];
?>
```

Output (in the browser): `Hello, Asha`

```
<form action="process.php" method="post">
  Password: <input type="password" name="pwd">
  <input type="submit">
</form>
```

```
<?php
  echo "Received password of length: " . strlen($_POST["pwd"]);
?>
```

Here the data travels in the **HTTP request body**, not the URL — it never appears in the address bar.

	GET	POST
Where data travels	Appended to the URL as a query string	In the HTTP request body
PHP superglobal	<code>\$_GET</code>	<code>\$_POST</code>
Visible to the user / bookmarkable	Yes	No
Data size limit	Limited (URL length, ~2048 chars in practice)	Effectively unlimited (server-config dependent)
Cached / stored in browser history	Yes	No
Suitable for	Search queries, filtering, bookmarkable links	Login forms, passwords, file uploads, sensitive data
Idempotent / safe to repeat	Yes (should not change server state)	No (may create/modify data — should not be silently repeated)

13.2 Sessions and cookies

	Cookie	Session
Where stored	Client's browser	Server (a session ID is stored client-side in a cookie or URL)

Typical use	Remember-me tokens, preferences, tracking	Login state, shopping cart, temporary per-visit data
Lifetime	Set explicitly (can persist across browser restarts)	Ends when the browser closes or the session times out, unless made persistent
Security	Visible/editable by the user; must not store sensitive data raw	More secure — the actual data stays server-side

```
<?php
    session_start();                // must be the very first thing PHP outputs
    $_SESSION["username"] = "Asha"; // store in the session
    echo $_SESSION["username"];     // retrieve it on this or a later request
?>
```

Output: Asha

```
<?php
    setcookie("username", "Asha", time() + 3600); // expires in 1 hour
    echo $_COOKIE["username"] ?? "not set";       // on a later page load: Asha
?>
```

Common error: `session_start()` must be called **before any HTML/output is sent** — calling it after `echo` or any whitespace before `<?php` produces the classic *"headers already sent"* error.

13.3 Form handling — the complete example

```
<!-- login.html -->
<form action="login.php" method="post">
    Username: <input type="text" name="user"><br>
    Password: <input type="password" name="pass"><br>
    <input type="submit" value="Login">
</form>
```

```
<?php
    // login.php
    $user = $_POST["user"] ?? "";
    $pass = $_POST["pass"] ?? "";
    if ($user == "admin" && $pass == "1234") {
        session_start();
        $_SESSION["user"] = $user;
        echo "Login successful";
    } else {
        echo "Invalid credentials";
    }
?>
```

Output (correct credentials): Login successful

Likely exam questions

1. Explain the difference between the GET and POST methods in HTML forms. **[10]** (*exactly the 2025 question*)
2. Differentiate between sessions and cookies in PHP, with examples. **[10]**

3. Write a PHP script that receives form data and validates a login. [10]
4. Why is HTTP called "stateless", and how do sessions/cookies solve that problem? [5]

MCQ traps

- Cookies are stored **client-side**; sessions are stored **server-side** (with only an ID cookie on the client) — this exact pair is the most common MCQ in this topic.
- GET data is visible in the URL and browser history — **never** put a password in a GET form.
- `session_start()` must be called before any output — a very common "basic PHP error" the syllabus explicitly names.

14. PHP + MySQL connection and common errors

Concept — in plain English first

A dynamic website's real value is reading/writing a database on every request. PHP connects to MySQL through the `mysqli` extension (or PDO); the pattern is always **connect** → **query** → **process result** → **close**.

Key points

```
<?php
    $conn = mysqli_connect("localhost", "root", "", "school_db");
    if (!$conn) {
        die("Connection failed: " . mysqli_connect_error());
    }

    $result = mysqli_query($conn, "SELECT name, marks FROM students");

    while ($row = mysqli_fetch_assoc($result)) {
        echo $row["name"] . " scored " . $row["marks"] . "<br>";
    }

    mysqli_close($conn);
?>
```

Output (for two rows in the table):

```
Asha scored 85
Toshi scored 90
```

The four `mysqli_connect()` arguments, in order: host, username, password, database name.

Common PHP errors and problems (the syllabus explicitly names this)

Error	Typical cause
Parse error: syntax error, unexpected ...	Missing semicolon, unmatched brace/quote

Undefined variable / undefined array key	Using <code>\$_GET/\$_POST</code> key that was not actually submitted; always check with <code>isset()</code>
Headers already sent	Output (even whitespace) sent before <code>session_start()</code> / <code>setcookie()</code> / <code>header()</code>
Connection failed	Wrong host/username/password/database name, or MySQL service not running
Call to undefined function	Extension (e.g. <code>mysqli</code>) not enabled in <code>php.ini</code> , or a typo in the function name
SQL injection vulnerability	Concatenating raw <code>\$_POST/\$_GET</code> values directly into a SQL query string instead of using prepared statements

A line worth writing in any "common errors" answer: always validate/sanitise input and prefer **prepared statements** (`mysqli_prepare` / PDO bound parameters) over string-concatenated queries — this is the standard defence against SQL injection, and ties this section back to database security (`CORE_06_DBMS.md`).

Likely exam questions

1. Write a PHP script to connect to a MySQL database and display all records from a table. **[10]**
2. List and explain common errors encountered in PHP scripting. **[5]**
3. How does SQL injection occur in a poorly written PHP script, and how is it prevented? **[5]**

MCQ traps

- `mysqli_connect()` argument order: **host, username, password, database** — a shuffled-order MCQ is common.
- "Headers already sent" is caused by output **before** `session_start()/header()`, not after.

15. Client-server HTTP request-response cycle

Concept — in plain English first

Every website interaction, underneath the HTML/PHP above, is one instance of this cycle repeated: the browser (client) asks for something, the server answers.

Key points

1. User types a URL / clicks a link / submits a form
2. Browser resolves the domain name to an IP address (DNS lookup)
3. Browser opens a TCP connection to the server (port 80/443)
4. Browser sends an HTTP request:
 - Request line: `GET /page.php HTTP/1.1` (or `POST`)
 - Headers: `Host`, `Cookie`, `Content-Type`, ...
 - Body: (form data, if `POST`)
5. Server receives the request, routes it (e.g. to the PHP interpreter for a `.php` file)
6. Server-side script executes (queries DB if needed), builds an HTML response
7. Server sends an HTTP response:
 - Status line: `HTTP/1.1 200 OK` (or `404`, `500`, `302`, ...)

Headers: Content-Type, Set-Cookie, ...

Body: the HTML page

8. Browser parses and renders the HTML/CSS/JS, and the connection may close or be reused (keep-alive)

Draw this as a simple two-box diagram — Client on the left, Server on the right, one arrow labelled "HTTP Request" going right, one arrow labelled "HTTP Response" going left, with the DNS lookup shown as a small side-box the client consults before the request arrow.

Common status codes	Meaning
200 OK	Success
301 / 302	Redirect (permanent / temporary)
404 Not Found	Resource does not exist
500 Internal Server Error	Server-side script/error

Likely exam questions

1. Describe the client-server request-response cycle for a web page. [5]
2. What happens between a user clicking a link and the page appearing in the browser? List the steps. [10]

MCQ traps

- DNS resolution happens **before** the HTTP request is sent, not as part of it.
- 404 = resource not found (client-side path error); 500 = server-side script failure — these are frequently swapped in MCQs.

Master cross-reference — where each syllabus phrase lives in this file

Syllabus phrase	Section
Factors in Developing Mobile Applications	§1
Frameworks and Tools in Mobile Applications	§2
Text-to-Speech Techniques	§3
Designing the Right UI	§4
Multichannel and Multimodal UIs	§5
Storing and Retrieving Data / Sync and Replication	§6
Android Storing and Retrieving Data	§7.5
Packaging and Deploying	§8
Phases of web site development (Implementation, Maintenance, Testing)	§9

Basic HTML Concepts / HEAD / TITLE / BODY / Paragraphs / Lists / Text / Hyperlink / Font / Image	§10
PHP: Server-side scripting, Installing PHP, Adding PHP to HTML, Syntax and Variables	§12
Passing information between pages, Basic PHP error/problems	§13, §14

● Forensic — email & digital evidence

Source: ComputerForensic/FOR_04_DigitalEvidenceAndMobile.md

FOR_04 — Digital Evidence, the Forensic Process, Mobile Computing & Mobile Forensics

Covers TP-II Unit 8 (Digital Evidences) and Unit 7 (Mobile Computing)

TP-II Unit 8 is the "**principles and process**" **capstone** of the elective — it takes everything from FOR_03 (imaging, hashing, recovery, tracing) and frames it as a disciplined, court-defensible **process** with named principles (ACPO), a named **model** (identify→...→present), and a deliverable (the **report** and **expert testimony**). Unit 7 is two halves welded together: the **mobile computing** engineering (cells, protocols, mobile databases, M-business) *and* **mobile forensics** (SIM/IMEI, CDR, acquisition levels, Android/iOS). The engineering half overlaps CORE networks; the forensic half is unique to this elective and high-yield.

Cross-references: imaging/hashing/write blockers/carving/signatures/steganalysis are treated in depth in **FOR_03 §5–§6** and file systems/registry/slack in **FOR_02**. Here they are **summarised and framed as process/evidence**, not repeated in full — the exam wants you to *place* each technique inside the process and the ACPO principles.

⚠ **Indian-law accuracy:** §65B IEA 1872 → §63 BSA 2023 (cite both); IT Act sections per FOR_03; flag any uncertain section number ⚠ *verify*. Never invent a section number or a case name.

PART A — DIGITAL EVIDENCE AND THE FORENSIC PROCESS (TP-II Unit 8)

§1. Digital Evidence — Concept and Characteristics

1.1 Concept

Digital evidence = any information of probative value that is stored or transmitted in binary form and may be relied upon in court. It is a **silent, invisible witness**: unlike a bloodstain you can hold, digital evidence must be *made visible* by tools, and it is uniquely **fragile, easily altered, easily copied, and time-sensitive**.

Characteristics (learn as a list — a common 5-mark question):

1. **Latent** — not visible to the naked eye; needs tools/software to be perceived.
2. **Fragile / volatile** — easily changed, damaged or destroyed (a single mount writes to it).
3. **Easily duplicated** — a perfect **bit-for-bit copy** is possible (and is the working method).
4. **Alterable and time-sensitive** — timestamps, RAM, logs change or vanish (order of volatility).

5. **Can transcend jurisdictions** — spread across devices, servers, countries.
6. **Voluminous** — terabytes; needs triage and selective analysis.
7. **Machine/format dependent** — needs the right tool to interpret.

Admissibility requirements (the four the examiner wants): digital evidence must be **Authentic** (it is what it claims to be — proven by hash + chain of custody), **Reliable/accurate** (sound tools and methods), **Complete** (not cherry-picked), and **legally obtained & admissible** (lawful seizure + **§65B IEA / §63 BSA certificate**). Some books add **"believable"** (understandable to the court). Cite the certificate requirement explicitly.

1.2 Digital-evidence storage devices (where it lives)

HDD, SSD, USB flash, SD/microSD, eMMC/UFS (phones), optical (CD/DVD/Blu-ray), magnetic tape (LTO), RAM/volatile memory, firmware/NVRAM, RAID/NAS/SAN, and **cloud**. (Geometry, SSD/TRIM problem and RAID levels are in **FOR_02 §2**; cloud in **FOR_03 §10**.)

1.3 Order of volatility (recap — see FOR_03 §4.3 for the full list + RFC 3227)

Most → least volatile: **registers/cache** → **routing/ARP/process tables & network connections** → **RAM** → **temp/swap files** → **disk** → **remote logs** → **physical topology** → **archival backups**. Collect most-volatile first. This is the operational form of the **Law of Progressive Change** (FOR_01 §1.5).

§2. Basic Rules / Principles for Cyber Forensics — the ACPO Principles

This is a near-certain question. Learn the **four ACPO principles** verbatim in substance. "ACPO" = the (UK) **Association of Chief Police Officers**; its *Good Practice Guide for Digital Evidence* is the most-cited statement of digital-forensic first principles worldwide.

#	Principle (substance)	What it means in practice
1	No action taken should change data held on a computer/storage medium that may be relied upon in court	Use write blockers ; image, don't work on the original; don't boot the evidence machine
2	If a person must access the original data, they must be competent to do so and able to explain the relevance and implications of their actions	Live acquisition (RAM, unlocked encryption) is allowed when justified by a competent examiner — document <i>why</i>
3	An audit trail (record of all processes applied) should be created and preserved; an independent third party should be able to repeat them and reach the same result	Contemporaneous notes, tool versions, hashes, chain of custody → reproducibility
4	The person in charge of the investigation has overall responsibility for ensuring these principles are followed	Accountability sits with the officer/lead

How to deploy in the exam: whenever a question asks "how do you ensure evidence is admissible / defensible", answer through the four ACPO principles + hashing + chain of custody + §65B/§63. It is a reusable spine for many 10/15-mark answers.

Related frameworks to name-drop: **RFC 3227** (evidence collection order), **NIST SP 800-86**

(guide to integrating forensics into incident response), **ISO/IEC 27037** (identification, collection, acquisition, preservation of digital evidence) Δ verify these document numbers before quoting them as exact.

§3. The Digital Forensic Process Model — the 6 phases (draw it)

Guaranteed descriptive question: "Explain the phases of the digital forensic process / cyber forensic investigation." Learn the six phases, one line each, plus what could go wrong at each.

IDENTIFICATION → PRESERVATION → COLLECTION → EXAMINATION → ANALYSIS → PRESENTATION
(find it) (protect it) (acquire it) (extract it) (interpret) (report/testify)
_____ documentation & chain of custody run through ALL phases

Phase	Goal	Key activities	Failure mode
1. Identification	Recognise what and where the evidence is	Identify devices, data sources, scope, legal authority; assess power state; identify volatile vs non-volatile	Missing a device (NAS, phone, cloud); scope creep beyond warrant
2. Preservation	Prevent any change	Isolate/secure scene, write blockers, order of volatility, imaging with hashing , chain of custody, Faraday for phones	Bootting the machine; no write blocker; remote wipe
3. Collection / Acquisition	Obtain the data	Bit-stream/logical/sparse imaging; live vs static; hash source & image	Wrong acquisition type; unverified image
4. Examination	Extract the relevant data from the mass	Recover deleted files, carve, decrypt, keyword search, filter with hash sets, parse artifacts	Overlooking slack/unallocated/ADS; missing hidden data
5. Analysis	Interpret and correlate → conclusions	Timeline reconstruction, link analysis, attribution, corroboration across sources	Over-claiming; confirmation bias (FOR_01 §3.4)
6. Presentation / Reporting	Communicate to the court	Clear report, exhibits, expert testimony , §65B/§63 certificate	Jargon; unsupported opinion; broken chain exposed in cross

Some models use different labels (**Kruse & Heiser: Acquire → Authenticate → Analyse**; **DFRWS: Identification → Preservation → Collection → Examination → Analysis → Presentation → Decision**; **NIST SP 800-86: Collection → Examination → Analysis → Reporting**). If asked for a model, give the **six-phase** version and mention that NIST condenses it to **four** (Collection, Examination, Analysis, Reporting). Naming one alternative model earns a mark.

§4. Data Acquisition Methods

4.1 By completeness

Method	What it captures	Use / trade-off
Physical (bit-stream) acquisition	Every sector — allocated + unallocated + slack + HPA/DCO	The forensic default; enables deleted-file recovery & carving; largest, slowest
Logical acquisition	Only the active files the file system reports	Faster/smaller; no deleted data ; used for huge systems or targeted, authorised subsets
Sparse acquisition	A logical copy of selected data + some deleted/unallocated fragments relevant to the case, without the whole device	Middle ground for very large data sets; capture only what is relevant (common in mobile/enterprise)
Targeted / triage	Specific artifacts (a few files, memory) captured quickly on scene	Live triage; used to decide priorities

4.2 By system state

	Live (dynamic) acquisition	Static (dead-box) acquisition
System	Running	Powered off
Gets	RAM, running processes, network state, unlocked encrypted volumes , cloud sessions	Full disk image, repeatable
Risk	Alters some state (justify per ACPO P2); rootkit may lie	Loses all volatile evidence

(Full treatment in **FOR_03 §4.4**; the modern standard is **RAM live first, then dead-box disk**.)

4.3 Imaging vs cloning; formats; write blockers; validation (recap)

- **Imaging** = create a *file* (raw/dd, **E01/EWF**, **AFF**) representing the device; **cloning** = disk-to-disk copy. Prefer imaging for evidence. (Formats: FOR_03 §5.3.)
- **Write blockers** (hardware = gold standard) prevent any write to the evidence. (FOR_03 §5.2.)
- **Data validation** = prove the acquired copy equals the source and stays unchanged, by **hashing** (MD5 128-bit / SHA-1 160-bit / SHA-256 256-bit; compute at acquisition, re-verify later; use two algorithms). (FOR_03 §5.4, incl. the collision/SSD-GC caveats.)

§5. Hash Algorithms, File Extensions & Signatures, Registry, Recovery, Disks

These are the "toolbox" facts Unit 8 lists; each is treated fully elsewhere — here is the exam-ready compression with pointers.

5.1 Hash values (see FOR_03 §5.4)

MD5 (128-bit), SHA-1 (160-bit), SHA-256 (256-bit). Hash = the digital **seal/fingerprint**; avalanche effect; MD5/SHA-1 collision-broken (still fine against accidental change; use SHA-256 or

dual hashes for defensibility). Uses: **integrity verification**, **de-duplication**, and **hash-set filtering** (NSRL known-good hashes to *exclude* OS files; known-bad hash sets to *flag* contraband like CSAM without opening files).

5.2 File extensions vs file signatures (see FOR_03 §6.3 for the full table)

Extension = renamable name; signature (magic bytes) = true type. Must-know headers: **JPEG** **FF D8 FF**, **PDF** **%PDF(25 50 44 46)**, **PNG** **89 50 4E 47**, **ZIP/Office** **PK(50 4B 03 04)**, **GIF** **GIF87a/GIF89a**, **EXE** **MZ(4D 5A)**, **ELF** **7F 45 4C 46**. Signature analysis detects **extension-mismatch masquerading**.

5.3 Windows Registry as evidence (see FOR_02 §5)

The registry is an **unintentional activity log**. High-value keys: **USBSTOR / MountedDevices / MountPoints2** (device history + user attribution), **UserAssist** (GUI program execution, ROT-13), **ShellBags** (folders browsed), **RecentDocs / RunMRU / TypedPaths / WordWheelQuery** (files, run commands, search terms), **Run/RunOnce/Services** (persistence), **TimeZoneInformation** (interpret all timestamps). Keys carry a **LastWrite** time; values do not. Cite this in any "registry as evidence" answer.

5.4 Data recovery techniques (see FOR_03 §6)

Undelete (metadata survives) → **carving** (content-based, no metadata) → **journal/shadow-copy recovery** (**\$LogFile**, **\$UsnJrnl**, **Volume Shadow Copies**). Deleted-file mechanics differ by FS (FAT **0xE5**; NTFS MFT unallocated; ext3/4 zero pointers). **SSD TRIM often destroys deleted data autonomously** (FOR_02 §2.2).

5.5 Windows disks, files and partitions — MBR vs GPT (a guaranteed compare)

Aspect	MBR (Master Boot Record)	GPT (GUID Partition Table)
Era / firmware	Legacy BIOS	UEFI
Location	First sector, LBA 0 (512 bytes): 446 B bootstrap + 64 B partition table (4×16) + 55 AA signature	GPT header at LBA 1 ; partition entries follow; a protective MBR at LBA 0 for backward compat
Max partitions	4 primary (or 3 primary + 1 extended → logical drives)	128 partitions typically (no extended/logical hack)
Max disk size	~2 TB (32-bit LBA × 512 B)	~9.4 ZB (64-bit LBA)
Redundancy	None (single MBR — a corrupt MBR loses the map)	Backup GPT header + table at end of disk ; CRC32 on header/table → integrity + recovery
Partition IDs	1-byte type code	GUID type + unique partition GUID

Forensic points

MBR is a single easily-imaged sector; **MBR bootkits**; recover the 4 entries; disk signature at offset 440

Examine backup GPT to detect tampering; recover deleted partitions from the backup table; **partition GUIDs** aid identification

MCQ nuggets: MBR = 512 bytes, **55 AA** boot signature, max 4 primary partitions, 2 TB limit; GPT = UEFI, 128 partitions, huge size, redundant + CRC-checked. The **partition table** lives in the MBR (LBA 0); GPT keeps a backup at the end of the disk.

Also relevant: **partitions vs volumes**, **slack space** (file/RAM/volume slack — FOR_02 §5.9), unallocated space, and **inter-partition/boot-record slack** as hiding places.

5.6 Email tracing, social-media analysis, steganalysis (see FOR_03 §6.5, §9, §1.5)

- **Email tracing:** read **Received:** headers **bottom→top** to the originating IP; check **SPF/DKIM/DMARC**, **From** vs **Return-Path/Message-ID/Reply-To**; then ISP legal process + §65B/§63 preservation. (Full walkthrough: FOR_03 §9.)
- **Social-media analysis:** documented open-source capture (URL + timestamp + hash) → platform legal process → device app-database forensics. (FOR_03 §1.5.)
- **Steganalysis:** statistical (chi-square, RS), signature, structure/size, comparison-to-original. (FOR_03 §6.5.)

§6. Forensic Report Writing and Expert Testimony

This closes the process (Phase 6). It is easy marks because it is a **list of qualities and contents** — and a CS graduate typically neglects it. Learn the report contents and the courtroom rules.

6.1 The forensic report — qualities and contents

Qualities (state these): accurate, objective/impartial, complete, clear (plain language), reproducible, defensible, and within the examiner's competence. Report facts and findings; keep opinion clearly separated and supported.

Standard contents (as a list — reproduce in the exam):

1. **Case/administrative details** — case/FIR number, requesting agency, examiner name & qualifications, dates.
2. **Authorisation & scope** — the warrant/authority and what was asked.
3. **Items received / chain of custody** — exhibit list, description, serial numbers, seal condition on receipt, **hash values**.
4. **Tools and methods** — hardware/software used **with versions**; write blocker used; SOP followed.
5. **Acquisition details** — type of image, source & image **hashes**, verification result.
6. **Findings** — the objective facts recovered (files, artifacts, timeline), with locations.
7. **Analysis / interpretation** — what the findings mean, correlations, timeline reconstruction.
8. **Conclusions / opinion** — clearly labelled as opinion, tied to findings, with limitations.
9. **Exhibits/appendices** — screenshots, extracted files, hash lists, logs.
10. **Declaration & signature** — including the **§65B IEA / §63 BSA certificate** for the electronic record.

6.2 Expert testimony (link to FOR_01 §3.3 duties of an expert)

- Expert opinion is admissible under **§45 Indian Evidence Act 1872 (now §39 BSA 2023 △ verify)**; it is **advisory — the court is not bound** and must apply its own mind.
- Reports of certain **Government Scientific Experts** may be used **without calling the expert** (old **CrPC §293**, now the corresponding **BNSS** provision △ verify number) — the GEQD, Chemical Examiner, FSL Director, etc. (FOR_01 §2.5).
- **The expert's primary duty is to the court**, not to the party that engaged them; testify **only within your field**; explain in plain language; disclose assumptions, limitations and anything detracting from the opinion; be ready to produce **case notes, tool logs, hashes and SOPs**.
- **Cross-examination targets**: the **chain of custody**, whether a **write blocker** was used, **hash verification**, tool validation, and examiner competence — which is exactly why the report must pre-answer all of them. **Never overstate** ("consistent with" vs "conclusively proves").

§6.3 Likely exam questions (Part A)

Marks	Question
5	List the characteristics of digital evidence.
5	State the ACPO principles of digital evidence.
5	Distinguish MBR and GPT.
5	Distinguish physical, logical and sparse acquisition.
5	What are the contents of a forensic report?
10	Explain the phases of the digital forensic process model with a diagram.
10	Discuss data acquisition methods (physical/logical/sparse, live/static) and when each is used.
10	Discuss forensic report writing and the duties of an expert witness in an Indian court.
15	Describe the complete lifecycle of digital evidence from identification to presentation, referencing ACPO principles, hashing, chain of custody and the §65B/§63 certificate.

MCQ traps (Part A)

- **ACPO has 4 principles**; Principle 1 = "no action should change the data".
- Process model (6): **Identification → Preservation → Collection → Examination → Analysis → Presentation**; **NIST condenses to 4** (Collection, Examination, Analysis, Reporting).
- **MBR max 4 primary partitions, ~2 TB, 55 AA; GPT 128 partitions, UEFI, backup table + CRC.**
- **Logical acquisition does not capture deleted data; physical (bit-stream) does.**
- **Expert opinion is advisory** and admissible under IEA §45 / BSA §39 (△ verify §39).
- Admissibility needs the **§65B IEA / §63 BSA certificate**.

PART B — MOBILE COMPUTING (TP-II Unit 7, engineering half)

This half overlaps CORE networks; keep it tight. The examiner wants the **cellular concepts** (cells, reuse, handoff), the **generation timeline (1G→5G)**, **GSM vs CDMA**, and **mobile**

databases / M-business. Then Part C turns to the forensic payoff.

§7. Mobile Connectivity and Cellular Concepts

7.1 Cells and the cellular idea

Concept: radio spectrum is scarce. Instead of one giant transmitter, the area is divided into small **cells**, each served by a **base station (BTS/Node B/eNodeB/gNodeB)** at low power, so the **same frequencies can be reused** in non-adjacent cells. This **frequency reuse** multiplies capacity — the founding idea of cellular telephony.

Term	Meaning
Cell	Geographic area served by one base station; drawn as a hexagon (tessellates without gaps/overlap)
Frequency reuse	Reusing the same channel set in cells far enough apart to avoid co-channel interference; reuse factor / cluster size (e.g. 3, 7)
Cell splitting / sectoring	Split a congested cell into smaller cells, or sector the antenna (e.g. 3×120°) to add capacity
Handoff / handover	Transferring an active call from one cell to the next as the user moves (hard handoff = break-before-make, GSM; soft handoff = make-before-break, CDMA)
Roaming	Using the network outside the home network's coverage
MSC / HLR / VLR	Mobile Switching Centre; Home & Visitor Location Registers — track subscriber location and route calls

Wireless delivery technologies & switching methods: early cellular used **circuit switching** (a dedicated channel for the call, like a landline). Data services moved to **packet switching** (GPRS/EDGE overlay on GSM), and modern networks (LTE/5G) are **all-IP packet-switched**, carrying even voice as packets (**VoLTE**). Switching evolution — *circuit* → *packet* → *all-IP* — is worth one sentence.

7.2 Cellular generations (1G → 5G) — the timeline

Gen	Era	Switching	Key technology	Data
1G	1980s	Circuit	Analog (AMPS)	Voice only
2G	1990s	Circuit (+packet overlay)	Digital — GSM (TDMA/FDMA) and CDMA (IS-95) ; SMS	~kbps; GPRS (2.5G) ~ up to ~114 kbps; EDGE (2.75G) ~up to ~384 kbps
3G	2000s	Packet + circuit	UMTS/WCDMA , CDMA2000 ; HSPA/HSPA+	~Mbps mobile broadband
4G	2010s	All-IP packet	LTE / LTE-Advanced (OFDMA); VoLTE	tens–hundreds of Mbps
5G	2019+	All-IP	NR (New Radio), mmWave, massive MIMO, network slicing	Gbps, ultra-low latency, massive IoT

7.3 GSM vs CDMA (a classic compare)

Aspect	GSM	CDMA
Access method	TDMA + FDMA (time + frequency slots)	Code Division — all share the band, separated by unique codes
Subscriber identity	SIM card — identity is portable between handsets	Historically handset-bound (no removable SIM in classic CDMA)
Handoff	Hard (break-before-make)	Soft (make-before-make)
Global reach	Dominant worldwide	Fewer markets
Forensic note	SIM is a discrete evidence item (see §9); IMSI/ICCID on the SIM	Subscriber data tied to the device; no SIM to seize (in classic CDMA)

7.4 Mobile information access devices, internetworking standards, WAP

- **Devices:** smartphones, tablets, feature phones, wearables, IoT/M2M modules, in-vehicle units.
- **Mobile data internetworking standards:** GPRS/EDGE (2G data), UMTS/HSPA (3G), LTE/5G-NR; local wireless **Wi-Fi (802.11)**, **Bluetooth**, **NFC**, **Zigbee**.
- **WAP (Wireless Application Protocol)** — the early stack for delivering web-like content to constrained phones: layered as **WAE / WSP / WTP / WTLS / WDP** over the bearer, with **WML** markup and a **WAP gateway** translating between WAP and HTTP. Largely historical (superseded by full mobile browsers) but examinable as a named stack. △ verify layer names before writing all five.

7.5 Mobile databases, tools/technology, and M-business

Mobile database — concept and needs: a database that runs on or synchronises with mobile devices under **intermittent connectivity, limited power/storage, and mobility**. Requirements: **local storage + replication/synchronisation** with a central server, **conflict resolution**, **caching, disconnected operation**, and **security/encryption** on a device that is easily lost.

Concept	Note
Replication & synchronisation	Local copy syncs with server when connected; conflict resolution (last-writer-wins, merge) needed
Caching / hoarding	Pre-fetch likely-needed data for offline use
Transaction models	Long-lived/disconnected transactions; optimistic concurrency suits mobility
On-device engines	SQLite (Android/iOS default), Realm; server side syncs to the enterprise DB
Forensic payoff	SQLite databases are where the evidence is on phones — SMS, call logs, contacts, chat apps, browser history all live in SQLite files (see §11)

M-business (mobile commerce): conducting commerce via mobile devices — mobile banking/payments (**UPI**, wallets), m-ticketing, location-based services, m-marketing. Enablers: secure mobile

payment, location awareness, ubiquity, personalisation. **Forensic relevance:** payment-app databases, UPI transaction records, location history — heavily used in fraud cases (FOR_03 §1.6).

§7.6 Likely exam questions (Part B)

Marks	Question
5	What is frequency reuse? Why are cells drawn as hexagons?
5	Distinguish hard and soft handoff.
5	Distinguish GSM and CDMA.
5	What is a mobile database and what challenges does mobility create?
10	Trace the evolution of cellular networks from 1G to 5G.
10	Explain mobile database synchronisation and its forensic significance.
15	Describe the architecture of a cellular mobile network (cells, base stations, MSC/HLR/VLR, handoff) and explain how mobility and switching evolved from 1G to 5G.

MCQ traps (Part B)

- **GSM uses TDMA/FDMA + SIM; CDMA uses code division, historically SIM-less.**
- **GPRS/EDGE are 2.5G/2.75G packet data; UMTS/WCDMA = 3G; LTE = 4G; NR = 5G.**
- **1G analog, 2G digital (+SMS).**
- **Cells are hexagonal** (tessellation, uniform coverage).
- **Soft handoff = CDMA (make-before-break); hard handoff = GSM.**
- **SQLite** is the default on-device mobile database.

PART C — MOBILE FORENSICS (TP-II Unit 7, forensic half)

§8. Mobile Forensics — Concept and Challenges

Concept: the science of recovering digital evidence from mobile devices under forensically sound conditions. A phone is a **richer, harder** exhibit than a PC: it holds calls, SMS, chats, photos (with GPS), app data, location history and health data — but it is **always-on, always-networked, encrypted by default, and endlessly varied** in OS/model.

Why it is harder than disk forensics:

- **Always connected** → risk of **remote wipe** / **new data overwriting** → **isolate immediately** (Faraday bag / airplane mode, keep charged).
- **Encryption by default** (modern Android/iOS full-disk/file-based encryption) → data unreadable without the passcode; **locked vs unlocked (AFU/BFU) state matters** hugely.
- **Proprietary, fast-changing** hardware/OS/connectors → tool support lags.
- **Volatile, hard to write-block** — you cannot easily attach a hardware write blocker to a soldered eMMC/UFS; acquisition inevitably interacts with the live device (document per ACPO P2).
- **State: BFU vs AFU — Before First Unlock** (keys not yet derived, very little accessible) vs

After First Unlock (keys in memory, far more data recoverable). Keep an AFU phone **powered and unlocked-alive** if lawful.

§9. SIM, Handset and Card Evidence; IMEI vs IMSI

9.1 The three evidence sources in a phone

Source	What it holds
Handset (internal memory)	The bulk: OS, apps, SQLite databases (SMS, call log, contacts, chat, browser), photos/videos with EXIF/GPS, location history, account tokens
SIM/USIM card	IMSI, ICCID , operator data, phonebook (ADN) , SMS (some stored on SIM) , last dialled numbers (LND), location info (LAI/TMSI)
Removable media (SD card)	Photos, videos, downloads, app data, sometimes backups — image separately with a write blocker

9.2 IMEI vs IMSI vs ICCID — memorise the distinction (guaranteed MCQ)

Identifier	Identifies	Length / format	Where it lives
IMEI (International Mobile Equipment Identity)	The handset/device	15 digits (TAC + serial + check digit); dial *#06# to display	In the phone hardware
IMSI (International Mobile Subscriber Identity)	The subscriber (the SIM/account)	up to 15 digits = MCC (3) + MNC (2–3) + MSIN	On the SIM/USIM
ICCID (Integrated Circuit Card ID)	The SIM card itself (serial number)	up to 19–20 digits	Printed on and stored in the SIM
MSISDN	The phone number		Network (HLR), not necessarily on the SIM

The one-liner: IMEI = the device; IMSI = the subscriber; ICCID = the SIM card; MSISDN = the number. Swap a stolen phone's SIM and the **IMEI stays** (traces the handset) while the **IMSI changes** — which is exactly how stolen-handset tracing works. **CEIR** (Central Equipment Identity Register) / the **sancharsaathi.gov.in** portal blocks stolen IMEIs in India △ verify name before quoting.

9.3 CDR analysis — Call Detail Records (a high-value topic)

Concept: a **CDR** is the **metadata record a telecom operator keeps for every call/SMS/data session** — *not* the content, but who/when/where/how long. Subpoena the operator; CDRs are a mainstay of Indian investigations (they place a suspect near a scene and map their network).

Typical CDR fields:

--

Field	Use
Calling & called numbers (A-party / B-party)	Who contacted whom
Date, time, duration	Timeline; pattern of life
Call type (voice/SMS/data), direction (MO/MT)	Nature of contact
IMEI	Which handset was used (catches SIM-swapping across handsets)
IMSI	Which SIM/subscriber
Cell ID / LAC + tower location, azimuth/sector	Approximate location of the phone at the time — the crux of CDR-based placing of a suspect
First/last cell	Movement during a call

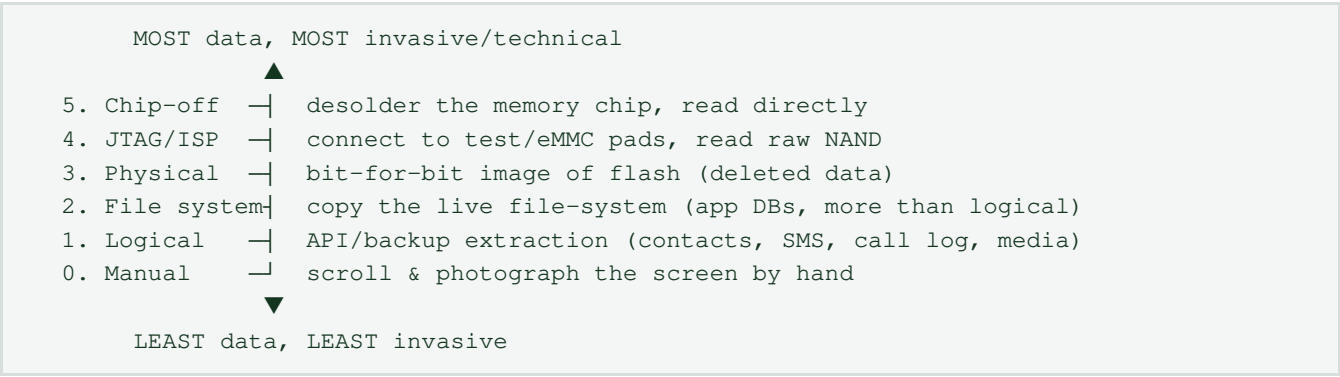
What CDR analysis achieves: place a suspect near the crime scene at the time (tower/cell location), establish contact between conspirators, build a **timeline and social-network graph**, detect **SIM-swap** (same IMEI, different IMSI) or **handset-swap** (same IMSI, different IMEI), and corroborate other evidence. **Limitations to state:** cell location is **approximate** (coverage area, not GPS), can be affected by tower load/handoff, and **CDR is metadata — content needs lawful interception** (IT Act §69) or on-device recovery.

9.4 Tower dump

A **tower dump** = all phones that connected to a given tower/cell in a time window — used to find an unknown suspect present at a scene. Privacy-sensitive; needs lawful authority.

§10. Mobile Acquisition Levels — the pyramid (learn the order)

A **classic question**: "Explain the levels of mobile data acquisition." Learn them from **least to most invasive / least to most complete** and note the trade-off (more data ⇔ more risk/skill).



Level	Method	Recovers	Notes / risk
-------	--------	----------	--------------

Manual	Operate the phone, photograph screens	Only what is visible	Changes state; no deleted data; last resort or quick triage
Logical	Use the OS/backup API (e.g. ADB backup, iTunes-style backup, vendor sync)	Contacts, SMS, call logs, media, some app data	Fast, safe, tool-supported; usually no deleted data ; needs the device unlocked
File system	Pull the device's file-system (databases, caches, config)	App SQLite DBs, logs, more artifacts including some "deleted" rows in SQLite (unvacuumed/WAL)	Often needs root/exploit/agent; richer than logical
Physical	Bit-for-bit image of the flash	Everything incl. unallocated/deleted (if not encrypted)	The goal; hard on modern encrypted phones
JTAG / ISP	Connect to JTAG test points or In-System Programming (eMMC pads) to read raw memory	Full physical image without desoldering	Advanced; needed for damaged/locked devices
Chip-off	Desolder the NAND/eMMC/UFS chip and read it in a reader	Raw physical memory	Destructive , expensive, expert-only; encryption still blocks reading without keys

Key modern caveat: on encrypted phones, **physical/JTAG/chip-off give you ciphertext** unless you also have the key (from an AFU state, a known passcode, or an exploit). So **logical/file-system acquisition from an unlocked/AFU device is often more valuable than a chip-off of a locked one**. Say this — it is the sophisticated point examiners reward.

SQLite recovery angle: even "deleted" chat/SMS rows often survive in **SQLite free-pages, WAL/journal files, and unvacuumed space** — a major reason file-system acquisition matters. Deleted rows are recoverable by carving the SQLite structures.

§11. Android and iOS Artifacts

Artifact	Android	iOS
Storage/FS	Historically ext4/F2FS; user data in <code>/data/data/<app>/</code>	APFS ; data under <code>/private/var/mobile/</code>
Encryption	FBE (file-based encryption), earlier FDE	Data Protection (per-file keys tied to passcode + Secure Enclave)
Contacts	<code>contacts2.db</code> (SQLite)	<code>AddressBook.sqlitedb</code>
SMS/MMS	<code>mmssms.db</code>	<code>sms.db</code>
Call log	<code>calllog.db</code> / <code>contacts2.db</code>	<code>call_history.db</code> / <code>CallHistory.storedata</code>
Chat apps	Per-app SQLite in <code>/data/data/<pkg>/databases/</code> (e.g. WhatsApp <code>msgstore.db</code> , often encrypted)	Per-app in the app container; WhatsApp <code>ChatStorage.sqlite</code>

Browser	Chrome SQLite (History)	Safari History.db
Photos/ media	DCIM + MediaStore ; EXIF incl. GPS	Photos library + Photos.sqlite ; EXIF/GPS
Location	Google location history, cell/Wi-Fi caches	cache.sqlite , frequent locations, KnowledgeC.db (device usage timeline)
Accounts/ tokens	accounts.db , app credentials	Keychain (Secure Enclave-protected)
Backups	ADB backup, cloud (Google)	iTunes/Finder backup (may be encrypted), iCloud
System usage	usagestats , logcat	KnowledgeC.db , powerlog , unified logs

The recurring theme: almost every phone artifact is a SQLite database. An Android/iOS answer that says "SMS in [mmssms.db/sms.db](#), calls in [callog.db/call_history.db](#), contacts, and app chat DBs are SQLite files recovered by file-system acquisition, with deleted rows carved from SQLite free-pages/WAL" scores well. ⚠ verify exact filenames per OS version before quoting them as absolute — they drift across versions; the **concept** (SQLite app DBs + EXIF/GPS + location DBs like KnowledgeC) is the examinable part.

§11.1 Likely exam questions (Part C)

Marks	Question
5	Distinguish IMEI, IMSI and ICCID.
5	What is a CDR? List its main fields.
5	What evidence can be recovered from a SIM card?
5	Why must a seized phone be placed in a Faraday bag?
5	Distinguish logical and physical acquisition of a mobile phone.
10	Explain the levels of mobile forensic acquisition from manual to chip-off.
10	Explain CDR analysis and how it places a suspect at a scene, with its limitations.
10	Describe the artifacts recoverable from an Android or iOS device.
15	Discuss the challenges of mobile forensics (encryption, connectivity, BFU/AFU state) and explain, with the acquisition-level pyramid, how you would forensically acquire a locked smartphone.

MCQ traps (Part C)

- **IMEI = equipment/handset (15 digits, *#06#); IMSI = subscriber (on SIM); ICCID = SIM serial.**
- **Same IMEI + different IMSI = SIM swap; same IMSI + different IMEI = handset swap.**
- **CDR is metadata (who/when/where), not call content.** Content needs §69 interception.
- **Chip-off is destructive; JTAG/ISP is non-destructive but advanced; both yield ciphertext if**

encrypted.

- **Faraday bag** prevents remote wipe / network changes.
- **Cell/tower location is approximate**, not GPS-precise.
- Most phone artifacts are **SQLite**; deleted rows survive in **WAL/free-pages**.

§12. Forensic Tools — the reference table (know 2–3 lines on each)

A "name and describe forensic tools" question is common, and every walkthrough answer is stronger if you name the right tool for the step. Learn the **category** → **tool** → **one-line use**.

Tool	Type / vendor	Primary use
EnCase	Commercial suite (OpenText/Guidance)	Full disk imaging (E01), analysis, keyword search, reporting; long courtroom pedigree; EnScript automation
FTK (Forensic Toolkit) + FTK Imager	Commercial (Exterro/AccessData)	Indexed searching, analysis; FTK Imager is the free, ubiquitous imaging/preview tool (raw/E01/AD1, memory capture)
Autopsy / The Sleuth Kit (TSK)	Open source (Brian Carrier)	GUI (Autopsy) over TSK CLI; file-system analysis, carving, timeline, keyword, hash sets — the standard free suite
X-Ways Forensics	Commercial (lightweight)	Fast, low-footprint disk analysis and imaging
Cellebrite UFED	Commercial (mobile)	Industry-standard mobile acquisition (logical/file-system/physical), lock bypass, app parsing
Magnet AXIOM / Oxygen Forensic Detective / MSAB XRY	Commercial	Mobile + computer + cloud artifact analysis; strong app/cloud parsing
Wireshark / tcpdump	Open source (network)	Packet capture/analysis (PCAP); protocol dissection; the network sniffer/analyser
Volatility / Rekall	Open source (memory)	Memory (RAM) forensics — processes, network, injected/fileless malware, keys
dd / dcfldd / dc3dd	Open source (imaging)	Command-line bit-stream imaging ; dcfldd/dc3dd add hashing & progress
HashCalc / md5sum / sha256sum / hashdeep	Utilities	Compute/verify hash values for integrity
RegRipper / Registry Explorer	Open source/free	Parse Windows registry hives into readable artifacts
PhotoRec / Scalpel / Foremost	Open source	File carving from unallocated space
Plaso / log2timeline	Open source	Super-timeline creation across many artifact types

Bulk Extractor	Open source	Scan images for emails, cards, URLs without parsing the FS
NSRL (RDS)	NIST hash set	Known-good hashes to filter out OS/software files
SANS SIFT / Kali / CAINE	Forensic Linux distros	Bootable toolkits bundling the above

Quick mapping to memorise: imaging → FTK Imager / dd / EnCase; disk analysis → Autopsy/ TSK, EnCase, FTK, X-Ways; memory → Volatility; network → Wireshark; mobile → Cellebrite UFED / XRY / AXIOM; registry → RegRipper; carving → PhotoRec/Scalpel; hashing → hashdeep/sha256sum; timeline → Plaso.

§12.1 Likely exam questions

Marks	Question
5	Name two tools each for disk, memory, network and mobile forensics.
5	What is FTK Imager used for?
10	Describe the major digital forensic tools and their uses in a table.
15	For a case involving a running PC and a locked smartphone, name the tools you would use at each stage (volatile capture, imaging, disk analysis, mobile acquisition, network, reporting) and justify each.

MCQ traps

- **Volatility = memory; Wireshark = network; Autopsy/Sleuth Kit = disk; Cellebrite UFED = mobile.**
- **FTK Imager (free) images/previews; the full FTK indexes/searches.**
- **The Sleuth Kit is the CLI; Autopsy is its GUI.**
- **EnCase's native image format is E01.**
- **NSRL hashes are used to **exclude** known files, not to find them.**

Appendix — verify-before-exam list for FOR_04

1. **ACPO** = 4 principles (safe). Framework doc numbers **NIST SP 800-86, ISO/IEC 27037, RFC 3227** — RFC 3227 safe; △ verify the NIST/ISO numbers before quoting exactly.
2. **§45 IEA 1872 → §39 BSA 2023** (expert opinion) and **CrPC §293 → BNSS** (government scientific experts) — △ verify the new section numbers; cite both codes.
3. **§65B IEA → §63 BSA** certificate — △ verify sub-section; cite both.
4. **IT Act §69** (interception) for call/data content — safe as concept; confirm sub-sections.
5. **CEIR / Sanchar Saathi (sancharsaathi.gov.in)** for IMEI blocking — △ verify name/URL.
6. **WAP stack layer names** (WAE/WSP/WTP/WTLS/WDP) — △ verify before writing all five.
7. **Android/iOS artifact filenames** (`mmssms.db`, `sms.db`, `calllog.db`, `call_history.db`, `KnowledgeC.db`, WhatsApp DB names) drift by version — △ verify; the SQLite concept is the marks.
8. **IMEI 15 digits, ICCID 19–20 digits, IMSI ≤15 (MCC+MNC+MSIN)** — safe, but double-check ICCID length wording.